Veermata Jijabai Technological Institute, Mumbai

Department of Computer Engineering

Artificial Intelligence – Project Report

Smart Disaster Response Planner

An AI Pathfinding Algorithm Visualizer for Disaster Rescue Scenarios

Submitted By	
Sakshi Vinayak Kalunge	241071032
Utkarsh Gupta	241070078
Manasa Gopisetty	241071039
Shreya Mane	241071041

Under the guidance and mentorship of Prof. Seema Shrawne

GitHub Repository: https://github.com/Ninjabeam20/AiProject

Table of Contents

1. Introduction 3
2. Problem Statement 4
3. Theory and Key Terminology 5
4. PEAS Representation 7
5. Approach to Logic 8
6. State Space Representation 9
7. Algorithms Used — Pseudocodes & Analysis 11
8. Algorithm Comparison 18
9. Best Algorithm and Justification 20
10. Implementation Details (Step by Step) 21
11. System Architecture and Components 23
12. Agent Decision-Making 24
13. Time and Space Complexity Analysis 26
14. Future Scope 29
15. Overall Observations 30
16. Results and Discussion 31
17. Conclusion 33
18. References 34

1. Introduction

1.1 Project Name

Smart Disaster Response Planner

1.2 What the Project Does

The Smart Disaster Response Planner is a real-time AI pathfinding visualizer built with Python and Pygame. It simulates a rescue vehicle navigating a grid-based disaster environment to locate and rescue victims of varying priority levels — HIGH (V1), MED (V2), and LOW (V3) — while avoiding impassable wall cells. The system implements eight distinct search algorithms from scratch, grouped into two categories: informed (heuristic-driven) and uninformed (blind) searches. Each algorithm is executed on the same environment and the tool records four performance metrics — path cost, nodes expanded, execution time, and peak memory usage — displayed in a live comparison panel so that users can directly contrast algorithm behaviour.

From a code perspective the project is structured into modular Python files. environment.py defines the 6×8 grid, cell types, walkability rules, victim tracking, and a BFS-validated random grid generator. heuristics.py provides three heuristic functions — Manhattan distance, priority-aware, and non-admissible. Each algorithm (astar.py, ida_star.py, best_first.py, ant_colony.py, branch_and_bound.py, bfs.py, dfs.py, dfid.py) follows a unified interface that returns a list of frame snapshots. main.py orchestrates the Pygame event loop, builds frames by dispatching to the selected algorithm, plays them back at the chosen speed, and records final metrics into the comparison panel. display.py handles all rendering.

1.3 Real-World Applicability

The problems this project simulates are directly analogous to real-world disaster response challenges. In earthquake, flood, or fire scenarios, rescue teams must navigate rubble-filled environments to reach survivors who have different survival probabilities (priority levels). The same questions the algorithms answer here — Which victim to rescue first? What is the shortest safe path? How does one balance optimality against speed? — are central to the design of autonomous search-and-rescue robots. The paper Intelligent Path Planning for Rescue Robot (Khanmohammadi & Soltani, 2011) formally demonstrates how combining multi-criteria decision-making with artificial potential fields allows a rescue robot to dynamically adjust its path depending on victim severity and environmental hazard levels. This project embodies a simplified, educational version of that concept using classical and modern graph search algorithms.

Concrete real-world use cases include: (1) Ground and aerial rescue robots deployed after natural disasters; (2) Autonomous navigation systems for emergency vehicles; (3) Hospital evacuation planning software; (4) Search-and-rescue drone path optimisation; and (5) Training simulators for rescue personnel to understand triage prioritisation.

2. Problem Statement

2.1 Formal Definition

Given a 6-row × 8-column grid-based environment E containing:

- One rescue base B at a known starting position,
- A set of victims V = $\{v_1, v_2, \ldots, v_n\}$ each labelled HIGH (V1), MED (V2), or LOW (V3),
- A set of walls W ⊆ E that are impassable,
- Empty cells through which the rescue vehicle may move in four cardinal directions (up, down, left, right),

Design and compare search algorithms that:

1. Find a path from B to each victim in priority order (informed) or nearest-first order (uninformed),
2. Minimise total path cost (total steps),
3. Minimise the number of nodes expanded (search effort),
4. Operate within practical memory and time bounds.

2.2 Constraints

- Diagonal movement is not permitted; only 4-directional movement allowed.
- Walls are permanently impassable; the vehicle cannot move through them.
- Once a victim is rescued, their cell becomes empty and the vehicle continues to the next target.
- The grid guarantees that all victims are reachable from the base (validated by BFS at generation time).
- Informed algorithms must respect priority ordering: all HIGH victims before MED, all MED before LOW.
- Uninformed algorithms have no knowledge of victim type or priority — they rescue nearest first.

3. Theory and Key Terminology

3.1 Search Problem

A search problem is defined by a state space, an initial state, one or more goal states, a set of actions or operators, a transition model describing the result of each action, and a path cost function. An agent solving a search problem generates a sequence of actions from the initial state to a goal state by systematically exploring the state space.

3.2 Key Terms

Term	Definition
Node	A single cell (row, col) in the grid. In A* each node also carries g, h, f values and a parent pointer for path reconstruction.
Open List	The set of nodes discovered but not yet expanded. Implemented as a priority queue (heap) for informed algorithms and a deque for BFS.
Closed Set	The set of cells already expanded. Prevents re-expansion and infinite loops.
Path Cost g(n)	The exact cost from the start node to the current node n. In this grid every edge costs 1, so g(n) equals the number of steps taken.
Heuristic h(n)	An estimated cost from node n to the nearest goal. In this project the default heuristic is Manhattan distance: $h(n) = \|r_1-r_2\| + \|c_1-c_2\|$.
Admissibility	A heuristic h is admissible if it never overestimates the true cost — i.e., $h(n) \leq h^*(n)$ for all n. Manhattan distance is admissible on a 4-connected grid with unit costs.
Optimality	An algorithm is optimal if it always finds a minimum-cost solution. A* and BFS (on unweighted grids) are optimal; DFS is not.
Completeness	An algorithm is complete if it is guaranteed to find a solution whenever one exists.
Informed Search	Uses a heuristic to guide expansion toward the goal. Examples: A*, IDA*, Best-First, Branch & Bound, ACO.
Uninformed Search	No heuristic; expands nodes without knowledge of goal location. Examples: BFS, DFS, DFID.
Priority Triage	Victim rescue is stratified: HIGH victims (V1) must all be rescued before MED (V2), and MED before LOW (V3). Mirrors real-world disaster triage (START, SALT protocols).
Manhattan Distance	The sum of absolute horizontal and vertical differences between two cells: $\|r_1-r_2\| + \|c_1-c_2\|$. It is the correct distance measure on a 4-connected grid without diagonals.
Pheromone Trail (ACO)	A virtual chemical marker deposited by simulated ants on visited edges. Higher pheromone concentration biases future ants toward that edge, reinforcing shorter paths.

f-threshold (IDA*)	The cost bound used by IDA* to prune branches. Starts at h(start) and is iteratively increased to the minimum f-value that exceeded the previous threshold.
Frame Snapshot	A dictionary capturing the full render state at one step: grid, vehicle position, visited cells, current path, rescued cells, f-values, log message, metrics.

4. PEAS Representation

PEAS (Performance measure, Environment, Actuators, Sensors) is the standard framework for characterising a rational agent. The table below formally describes the rescue agent in this project.

Component	Description
Performance Measure	(1) Minimise total path cost (total steps taken to rescue all victims) (2) Minimise nodes expanded (search effort / computation) (3) Rescue all HIGH priority victims before MED, and MED before LOW (triage compliance) (4) Minimise execution time and peak memory usage (5) Ensure 100% victim rescue rate — no victim left unreachable.
Environment	A 6-row × 8-column discrete grid Cell types: BASE (B) — start; VICTIM_HIGH (V1, red); VICTIM_MED (V2, orange); VICTIM_LOW (V3, green); WALL (X, impassable); EMPTY (0, passable) Grid is static (walls do not move) Deterministic: every action has a single predictable outcome Fully observable from an algorithmic perspective — the algorithm has access to the complete grid structure Single-agent: only one rescue vehicle Guaranteed connectivity: all victims reachable from base (validated by BFS at grid generation time).
Actuators	Move Up — decrements row index by 1 Move Down — increments row index by 1 Move Left — decrements column index by 1 Move Right — increments column index by 1 Rescue — marks victim cell as EMPTY increments rescued counter; records position in rescued set.
Sensors	Full grid state access via Environment.grid[r][c] Victim detection: Environment.get_victims() returns list of (row, col, type) Walkability check: Environment.is_walkable(r, c) — returns False for walls and out-of-bounds Neighbour expansion: Environment.get_neighbours(r, c) — returns valid 4-directional adjacent cells Base position: Environment.base Heuristic estimate: heuristics.heuristic_manhattan(r, c, victims) — Manhattan distance to nearest victim.

4.1 Agent Type Classification

Property	Value / Description
Observability	Fully Observable (complete grid visible to agent)
Determinism	Deterministic (actions have single, predictable outcomes)
Episodic vs Sequential	Sequential (each rescue changes state for future decisions)
Static vs Dynamic	Static (environment does not change while agent deliberates)
Discrete vs Continuous	Discrete (finite grid cells, finite actions)
Single vs Multi-Agent	Single Agent

5. Approach to Logic

5.1 Overall Design Philosophy

The project adopts a separation-of-concerns architecture: the environment model is completely decoupled from the search algorithms, which are in turn decoupled from the rendering engine. Every algorithm works on a clone of the environment (env.clone()) so that its internal modifications — rescuing victims, updating the grid — never affect the original state or other algorithms.

5.2 Unified Algorithm Interface

All eight algorithms implement the same interface:

```
class AlgorithmSearch:
    def __init__(self, env):
        self.env = env.clone()   # Always clone — never mutate original
    def run(self) -> tuple[list, dict, list]:
        # Returns: (frames, metrics, log)
        ...
```

The run() method returns three things: a list of frame snapshots for animation playback, a metrics dictionary, and a human-readable log. This unified contract means display.py renders every algorithm identically — no algorithm-specific rendering code.

5.3 Priority-Aware vs. Nearest-First

Informed algorithms implement a three-tier priority loop. Before expanding any node they check: are there any V1 victims left? If yes, search only for V1 victims. Only when no V1 remains do they search for V2, and only when no V2 remains do they search for V3. This mirrors real disaster triage where critically injured patients receive attention before less critical ones.

Uninformed algorithms pass target_types=('V1','V2','V3') to their core search function — they find the first reachable victim regardless of type because, by definition, they have no heuristic knowledge to distinguish between victim types.

5.4 Frame-Based Animation

Rather than animating algorithms in real time (which would couple rendering speed to algorithm speed), each algorithm pre-computes its entire execution as a list of frame dictionaries. The Visualizer then plays back these frames at the user-selected speed (125–2000 ms/frame). This architecture allows instant algorithm replay, speed control, and algorithm comparison without re-running the search.

5.5 Memory Measurement

Peak memory is measured using Python's built-in tracemalloc module, started immediately before algorithm.run() and stopped after. This captures all heap allocations made during the search, including the frame list, visited sets, priority queues, and path dictionaries.

6. State Space Representation

6.1 State Definition

A state S is fully described by the tuple:

S = (position, rescued_set, grid_state)

- position = (row, col) — current cell of the rescue vehicle, 0 ≤ row ≤ 5, 0 ≤ col ≤ 7.
- rescued_set — a frozen set of 'row,col' strings for victims already rescued.
- grid_state — the 6×8 grid matrix (V1/V2/V3 cells become '0' after rescue).

6.2 Initial State

S_0 = (base_position, ∅, full_grid) — the vehicle starts at the base, no victims have been rescued, and the grid is in its initial configuration.

6.3 Goal State

S_goal = (any_position, {all_victim_positions}, empty_victim_grid) — all victims have been rescued. The vehicle does not need to return to base.

6.4 Actions / Operators

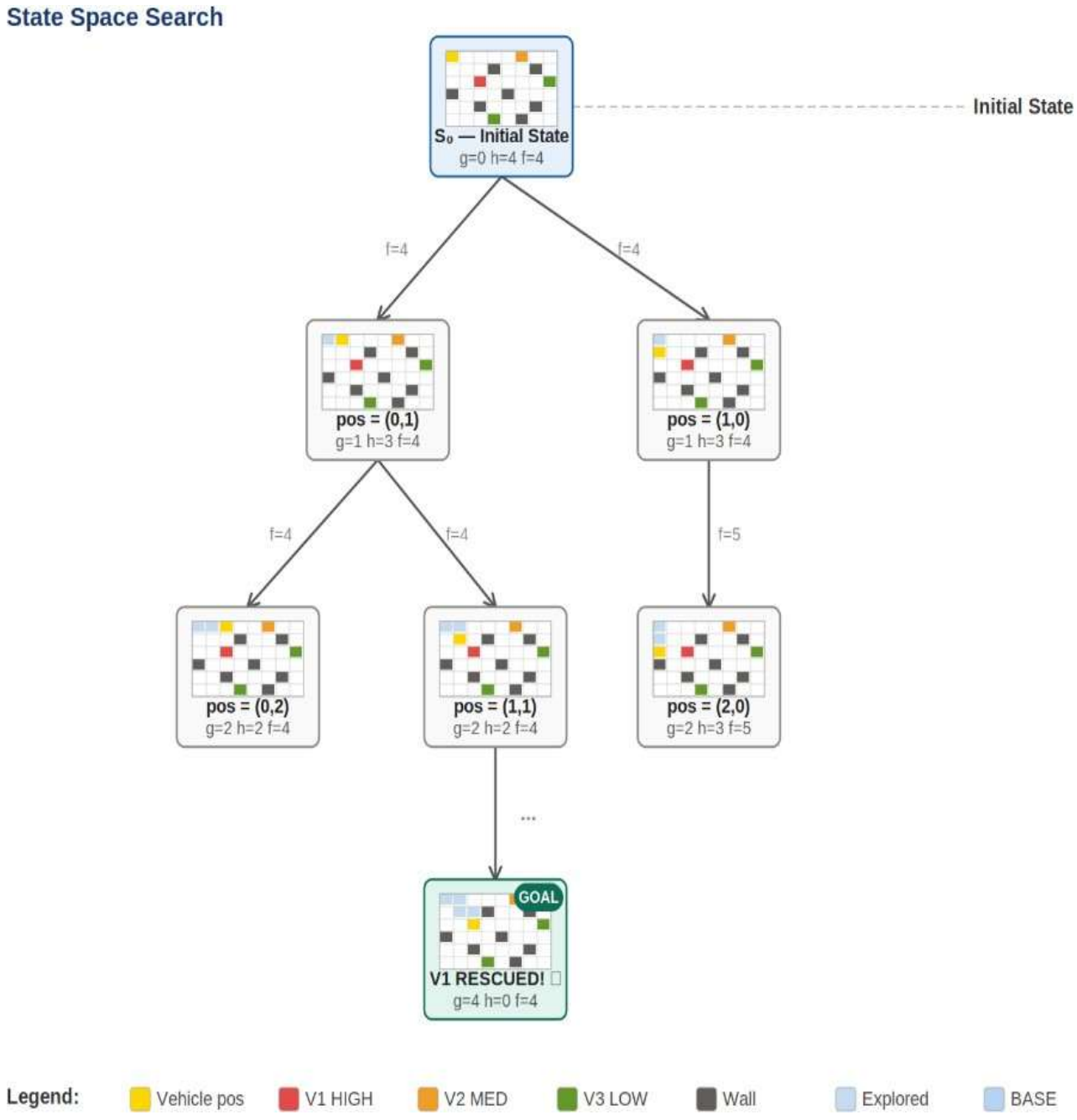
Action	Precondition	Effect
Move Up	row > 0 AND grid[row-1][col] ≠ X	position ← (row-1, col)
Move Down	row < 5 AND grid[row+1][col] ≠ X	position ← (row+1, col)
Move Left	col > 0 AND grid[row][col-1] ≠ X	position ← (row, col-1)
Move Right	col < 7 AND grid[row][col+1] ≠ X	position ← (row, col+1)
Rescue	grid[row][col] ∈ {V1,V2,V3}	grid[row][col] ← '0'; rescued_set ← rescued_set ∪ {key}

6.5 State Space Size

The 6×8 = 48-cell grid has up to 48 positions × 2^(number of victims) victim configurations. With 3–5 victims the theoretical state space upper bound is 48 × 2^5 = 1,536 states per rescue pass. Since the project processes victims one at a time (each search finds the next single victim), the effective search space per call is just 48 nodes (all grid cells), making it highly tractable.

6.6 State Space Graph

The following represents a simplified portion of the state space graph for a 3-victim grid. Each node is labelled with the vehicle position and victims remaining. Edges represent single-step movements.



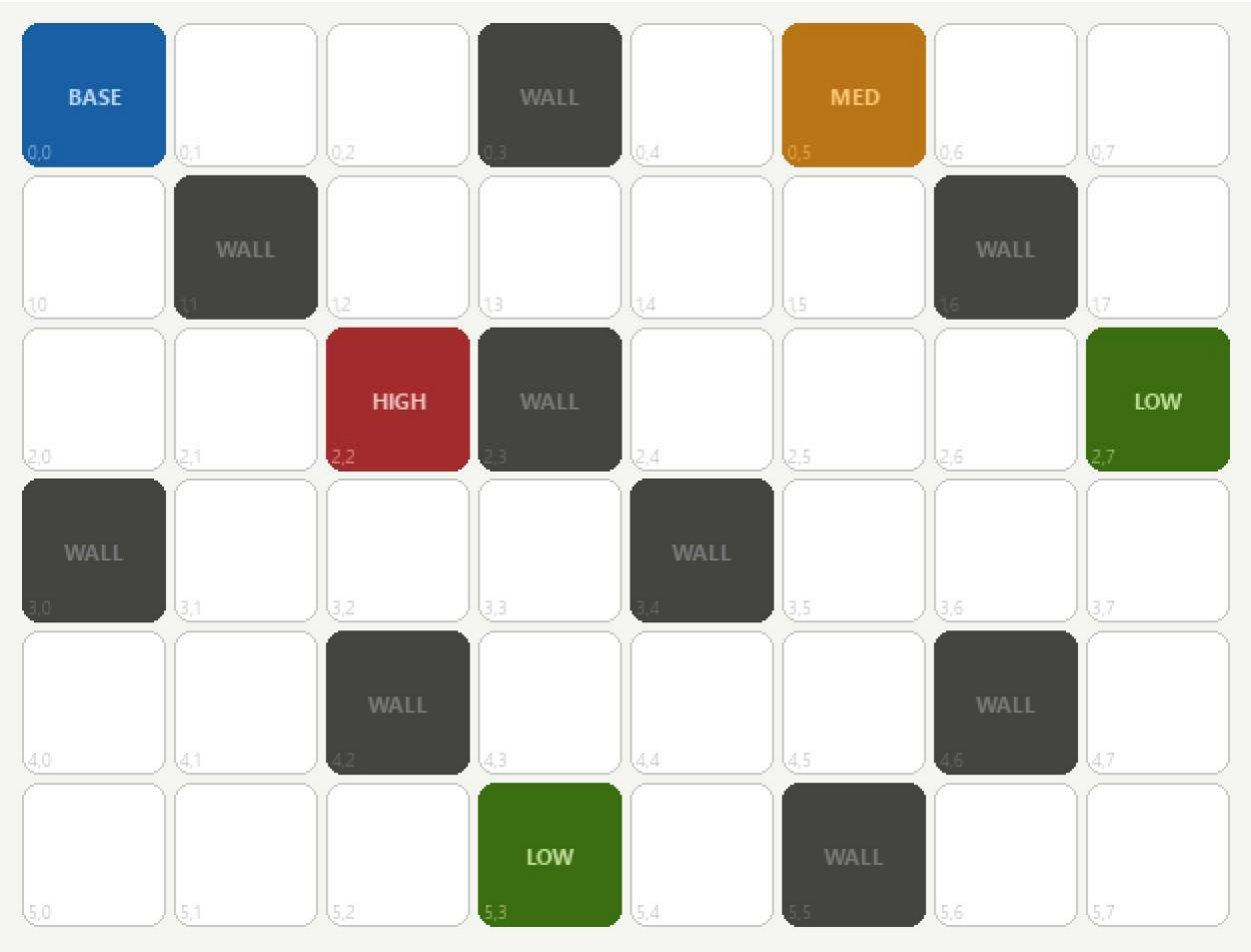


Figure 1 — Default 6×8 Grid Layout with BASE (blue), V1 (red), V2 (orange), V3 (green), and walls (dark grey).

7. Algorithms Used — Pseudocodes & Analysis

All eight algorithms follow the same outer loop: while victims remain, call the core search function, animate the path, rescue the victim, and repeat. The inner search function differs per algorithm. Pseudocodes below reflect the actual implementation.

7.1 Breadth-First Search (BFS)

Category: Uninformed

BFS expands nodes level by level using a FIFO queue. It finds the shortest path in terms of number of edges (steps) in an unweighted graph.

```
BFS(start, target_types, rescued):
  queue  ← deque([(start)])
  visited ← {start_key}
  parent  ← {start_key: None}
  while queue not empty:
    (r, c) ← queue.popleft()
    nodes_expanded += 1
    if grid[r][c] ∈ target_types AND key ∉ rescued:
      return reconstruct_path(parent, key), nodes_expanded, visited
    for (nr, nc) in get_neighbours(r, c):
      if nkey ∉ visited:
        visited.add(nkey)
        parent[nkey] ← key
        queue.append((nr, nc))
  return None
```

Metric	Value
Time Complexity	O(V + E) = O(48 + 4×48) ≈ O(N²) where N = grid size
Space Complexity	O(V) = O(48) for visited set, queue, and parent dict
Optimal?	Yes — shortest path guaranteed (unit-cost edges)
Complete?	Yes — guaranteed to find solution if one exists
Purpose in Code	Nearest-first rescue (uninformed). Also used in generate_random_grid() to validate grid connectivity.

7.2 Depth-First Search (DFS)

Category: Uninformed

DFS explores as deep as possible along each branch using a LIFO stack before backtracking. It is not optimal — the path found depends on neighbour ordering.

```
DFS(start, target_types, rescued):
  stack   ← [{'r':start.r, 'c':start.c, 'path':[]}]
  visited ← {}
  while stack not empty:
    node ← stack.pop()
    r, c ← node.r, node.c
    if key ∈ visited: continue
    visited.add(key)
    nodes_expanded += 1
    if grid[r][c] ∈ target_types AND key ∉ rescued:
      return full_path, nodes_expanded, visited
    for (nr, nc) in get_neighbours(r, c):
      if nkey ∉ visited:
        stack.append({'r':nr,'c':nc,'path':node.path + [key]})
  return None
```

Metric	Value
Time Complexity	O(V + E) worst case — may visit all reachable cells
Space Complexity	O(V) for stack and visited set (stores full path in each stack entry — actual overhead $O(V^2)$ worst case)
Optimal?	No — path found is first found, not shortest
Complete?	No — may loop infinitely without visited set (with visited set: complete on finite graphs)
Purpose in Code	Nearest-first rescue (uninformed). Demonstrates non-optimal behaviour for comparison.

7.3 Depth-First Iterative Deepening (DFID)

Category: Uninformed

DFID runs depth-limited DFS repeatedly with increasing depth limits (0, 1, 2, …) until the goal is found. It combines BFS's optimality with DFS's linear space complexity.

```
DFID(start, target_types, rescued):
  for depth_limit = 0 to max_depth:
    result ← DLS(start, 0, depth_limit, path=[], target_types, rescued)
    if solution found: return path, nodes_expanded
    if no cutoff occurred: return None    // unreachable

DLS(r, c, depth, limit, path, target_types, rescued):
```

```
  if depth > limit: return CUTOFF
  if grid[r][c] ∈ target_types AND key ∉ rescued: return SUCCESS
  cutoff_occurred ← False
  for (nr, nc) in get_neighbours(r, c):
    if nkey ∉ visited_stack:
      result ← DLS(nr, nc, depth+1, limit, path+[nkey], ...)
      if result == SUCCESS: return SUCCESS
      if result == CUTOFF: cutoff_occurred ← True
  return CUTOFF if cutoff_occurred else FAILURE
```

Metric	Value
Time Complexity	O(b^d) where b = branching factor (≤4), d = solution depth. On a 48-cell grid this is bounded by $O(N^2)$.
Space Complexity	O(d) = O(depth) — only the current path is stored. Empirically much lower memory than BFS.
Optimal?	Yes — finds shallowest solution (equivalent to BFS on unit-cost graphs)
Complete?	Yes — on finite acyclic graphs with visited stack
Purpose in Code	Demonstrates optimal memory-efficient uninformed search. Two-phase implementation: Phase 1 finds solution depth without emitting frames; Phase 2 replays with frame recording.

7.4 A* Search

Category: Informed

A* selects the node with minimum f(n) = g(n) + h(n) from a min-heap, where g is the actual cost from start and h is the admissible heuristic (Manhattan distance). It is both optimal and complete with an admissible heuristic.

```
A*(start, victims, rescued, target_types):
  heap ← [(f0, counter, start.r, start.c)]
  visited ← {start_key}
  g_cost  ← {start_key: 0}
  parent  ← {start_key: None}
  while heap not empty:
    (f, _, r, c) ← heappop(heap)
    g ← g_cost[key]
    nodes_expanded += 1
    if grid[r][c] ∈ target_types AND key ∉ rescued:
      return reconstruct_path(parent, key), nodes_expanded, visited
    for (nr, nc) in get_neighbours(r, c):
      new_g ← g + 1
      if nkey ∉ visited:
        visited.add(nkey)
        g_cost[nkey] ← new_g
        parent[nkey] ← key
```

```
        h ← heuristic(nr, nc, victims)
        heappush(heap, (new_g + h, counter, nr, nc))
  return None
```

Metric	Value
Time Complexity	O(E log V) — each node is inserted into the heap at most once; heap operations are O(log V)
Space Complexity	O(V) — stores visited set, g_cost dict, parent dict, and heap, each bounded by O(48)
Optimal?	Yes — with admissible heuristic (Manhattan distance is admissible on a 4-connected grid)
Complete?	Yes
Purpose in Code	Gold standard for informed search. Rescues victims in strict priority order (V1 → V2 → V3).

7.5 IDA* (Iterative Deepening A*)

Category: Informed

IDA* combines A*'s heuristic guidance with iterative deepening's linear space usage. It performs depth-first searches with an f-cost threshold, increasing the threshold to the minimum f-value that exceeded it in the previous iteration.

```
IDA*(start, victims, rescued, target_types):
  threshold ← h(start, victims)
  loop:
    result ← SEARCH(start, g=0, threshold, path=[start_key])
    if result == FOUND: return path, nodes_expanded, all_visited
    if result == ∞: return None
    threshold ← result

SEARCH(r, c, g, threshold, path, ...):
  f ← g + h(r, c, victims)
  if f > threshold: return f
  nodes_expanded += 1
  if grid[r][c] ∈ target_types AND key ∉ rescued:
    found ← path; return FOUND
  min_exceeded ← ∞
  for (nr, nc) in get_neighbours(r, c):
    if nkey ∉ visited_stack:
      result ← SEARCH(nr, nc, g+1, threshold, path+[nkey])
      if result == FOUND: return FOUND
      min_exceeded ← min(min_exceeded, result)
  return min_exceeded
```

Metric	Value
Time Complexity	O(b^d) where d = optimal depth. On a 48-cell grid, multiple DFS iterations are run but each is bounded by O(V).
Space Complexity	O(d) — recursive stack only; far more memory-efficient than A*
Optimal?	Yes — with admissible heuristic
Complete?	Yes
Purpose in Code	Memory-efficient alternative to A*. Preferred when RAM is constrained (e.g., embedded rescue robots).

7.6 Greedy Best-First Search

Category: Informed

Best-First Search expands the node with the lowest h(n) value, ignoring g(n) entirely. It is fast but not optimal — it can be misled by the heuristic toward a suboptimal path.

```
BestFirst(start, victims, rescued, target_types):
  heap ← [(h₀, counter, start.r, start.c)]
  visited ← {start_key}
  parent  ← {start_key: None}
  while heap not empty:
    (h, _, r, c) ← heappop(heap)    // NOTE: no g(n) component
    nodes_expanded += 1
    if grid[r][c] ∈ target_types AND key ∉ rescued:
      return reconstruct_path(parent, key), nodes_expanded, visited
    for (nr, nc) in get_neighbours(r, c):
      if nkey ∉ visited:
        visited.add(nkey)
        parent[nkey] ← key
        heappush(heap, (h(nr,nc,victims), counter, nr, nc))
  return None
```

Metric	Value
Time Complexity	O(E log V) in best case; O(V²) in worst case (poor heuristic)
Space Complexity	O(V) for heap, visited, and parent
Optimal?	No — path cost g(n) is ignored
Complete?	Yes (with visited set on finite graphs)
Purpose in Code	Fastest informed search — typically expands fewest nodes but produces longer paths than A*.

7.7 Branch and Bound (B&B)

Category: Informed

Branch and Bound performs an exhaustive search guided by an upper-bound cost. It maintains the best solution found so far and prunes any branch whose current cost already equals or exceeds that bound. Unlike A*, it does not use f = g + h for prioritisation — it uses g alone for pruning.

```
BranchAndBound(start, victims, rescued, target_types):
  heap ← [(h0, counter, node{r,c,g=0,path=[]})]
  best_cost ← ∞
  best_path ← None
  visited_best ← {}    // best g seen per cell
  while heap not empty:
    (_, _, node) ← heappop(heap)
    r, c, g ← node.r, node.c, node.g
    if g >= best_cost: continue    // prune
    if visited_best[key] <= g: continue  // prune
    visited_best[key] ← g
    nodes_expanded += 1
    if grid[r][c] ∈ target_types AND key ∉ rescued:
      best_cost ← g
      best_path ← reconstruct(node.path + [(r,c)])
      continue  // keep searching for better paths
    for (nr,nc) in get_neighbours(r,c):
      if visited_best[nkey] > g+1:
        heappush(heap, (g+1+h(nr,nc), counter, child_node))
  return best_path, nodes_expanded, all_visited
```

Metric	Value
Time Complexity	O(E log V) average with good pruning; worst case $O(V^2)$
Space Complexity	O(V) for heap, visited_best dict, and node paths
Optimal?	Yes — continues searching after first solution to confirm optimality
Complete?	Yes
Purpose in Code	Demonstrates exhaustive-but-pruned search. Expands more nodes than A* (no early exit on first goal) but guarantees optimality via cost bounding.

7.8 Ant Colony Optimisation (ACO)

Category: Informed (Probabilistic)

ACO simulates the foraging behaviour of real ants. Simulated ants lay virtual pheromone trails on edges they traverse. Shorter paths accumulate more pheromone per iteration, biasing subsequent ants toward those paths. Over many iterations the population converges on the near-optimal path.

```
ACO(start, victims, rescued, target_types, iterations=50, ants=10):
  pheromone[edge] ← 1.0  // initialise uniformly
  best_path ← None
  best_length ← ∞
  for iter in range(iterations):
    all_ant_paths ← []
    for ant in range(num_ants):
      path ← construct_ant_path(start, target_types, pheromone, heuristic)
      all_ant_paths.append(path)
      if len(path) < best_length:
        best_length ← len(path)
        best_path ← path
    pheromone ← (1 - evaporation) × pheromone  // evaporate
    for path in all_ant_paths:
      for edge in path:
        pheromone[edge] += 1.0 / len(path)  // reinforce
  return best_path, nodes_expanded, all_visited

construct_ant_path(start, ...):
  visited ← {start}
  pos ← start
  while target not reached AND steps < max_steps:
    neighbours ← unvisited walkable neighbours of pos
    if empty: break
    weights ← pheromone[edge]^alpha × (1/h(n))^beta  for each neighbour
    next ← random.choices(neighbours, weights=weights)
    path.append(next)
    pos ← next
```

Metric	Value
Time Complexity	O(iterations × ants × V) = O(50 × 10 × 48) per victim — effectively $O(N^2)$ per victim
Space Complexity	O(E) for pheromone matrix (E = edges ≈ 4×48 = 192)
Optimal?	No — probabilistic; converges to near-optimal but not guaranteed
Complete?	No — may fail to find a path if max_steps is too small or pheromone converges poorly
Purpose in Code	Only nature-inspired / meta-heuristic algorithm. Demonstrates trade-off between computation (many iterations) and solution quality (near-optimal but not exact).

8. Algorithm Comparison

The table below compares all eight algorithms across the most important theoretical and practical dimensions.

Algorithm	Optimal	Complete	Notes / Trade-offs
A*	Yes	Yes	Gold standard. Balances path cost and heuristic. Fewest nodes expanded among optimal algorithms. Requires O(V) memory.
IDA*	Yes	Yes	Memory-efficient A*. Iterates over depth thresholds — pays with redundant node re-expansion but uses only O(d) space. Ideal for memory-constrained environments.
Best-First	No	Yes	Fastest to find a goal due to pure heuristic guidance. May produce suboptimal paths when heuristic is misleading. Useful when speed > optimality.
B&B	Yes	Yes	Exhaustive with pruning. Does not exit on first goal — verifies optimality by continuing search. More nodes expanded than A* but rigorously correct.
ACO	No	No	Probabilistic convergence. High computational cost (50 iter × 10 ants). Non-deterministic — results vary between runs. Best for dynamic or large-scale environments.
BFS	Yes	Yes	Uninformed shortest path. Explores all nodes level-by-level. High memory usage at large depths. Optimal for unit-cost unweighted grids.
DFS	No	No*	Fast node discovery but highly non-optimal paths. Memory-efficient (stack-based). (*Complete with visited set on finite graphs.)
DFID	Yes	Yes	Best uninformed search overall. Combines BFS optimality with DFS space efficiency. Slightly more computation due to repeated iterations.

COMPARISON 8 runs

Algo	Cost	Nodes	Time	Mem
A*	20	43	10.36ms	111.0k
IDA*	20	33	12.93ms	98.1k
Best-Fir	24	37	10.01ms	112.8k
ACO	20	112	1017.1ms	237.8k
B&B	20	68	14.29ms	111.2k
BFS	19	75	9.77ms	110.5k
DFS	46	51	19.03ms	245.7k
DFID	19	299	39.74ms	308.6k

Figure 2 — Live Algorithm Comparison Panel showing Path Cost, Nodes Expanded, Execution Time, and Memory Usage across all 8 algorithms.

8.1 Informed vs Uninformed — Conceptual Comparison

Dimension	Informed	Uninformed
Priority Handling	Strict triage: V1 → V2 → V3	Nearest first — no priority knowledge
Node Selection	h(n) or f(n) guides expansion toward goal	Blind: FIFO (BFS), LIFO (DFS), or depth-limited
Nodes Expanded	Fewer (heuristic prunes poor branches)	More (no pruning — blind exploration)
Path Cost	Optimal (A*, IDA*, B&B) or near-optimal (Best-First)	Optimal (BFS, DFID) or suboptimal (DFS)
Memory	O(V) for A*/Best-First/B&B; O(d) for IDA*	O(V) for BFS; O(d) for DFS/DFID
Real-world suitability	High — prioritises critical victims	Moderate — may rescue low-priority victims first

9. Best Algorithm and Justification

9.1 Verdict: A* is the Best Overall Algorithm for This Project

A* consistently produces the optimal (minimum step count) path while expanding the fewest nodes among all optimal algorithms. It achieves this through its f(n) = g(n) + h(n) evaluation function, which balances the known cost from the start against the estimated remaining cost, directing the search precisely toward the goal without unnecessary exploration.

9.2 Why Not the Others?

Algorithm	Why Not Best?
IDA*	Optimal and memory-efficient but re-expands nodes across iterations — slower execution time than A* on a small 48-cell grid where memory is not a constraint.
Best-First	Fastest to find a victim but not optimal — produces longer paths, which is unacceptable in time-critical rescue scenarios.
B&B	Also optimal but expands more nodes than A* because it continues searching after the first goal to verify optimality. Higher nodes_expanded than A* on this grid.
ACO	Non-deterministic and computationally expensive (50 iterations). Suitable for large dynamic environments but overkill and imprecise for a 48-cell static grid.
BFS	Optimal uninformed search but expands all nodes level-by-level with no heuristic guidance — explores far more cells than A*.
DFS	Neither optimal nor complete (without visited set) — produces significantly longer paths. Unsuitable for rescue prioritisation.
DFID	Optimal and memory-efficient uninformed search, but slower than BFS on this grid due to repeated iteration overhead.

9.3 Real-World Justification

In the context of the paper Intelligent Path Planning for Rescue Robot (Khanmohammadi & Soltani, 2011), the authors combine multi-criteria decision-making to compute victim priority and Artificial Potential Fields (APF) for path generation. APF is analogous to a greedy best-first approach guided by potential energy. However, APF suffers from local minima. A* — with its global optimality guarantee and admissible heuristic — avoids local minima entirely, making it the superior choice for the structured grid environment in this project. Similarly, the terrain-enhanced A* described in the PMC paper on disaster rescue path planning (2022) confirms that A* with augmented cost functions outperforms uninformed alternatives in real-world rescue scenarios.

10. Implementation Details

10.1 Grid Initialisation

- environment.py defines DEFAULT_GRID — a 6×8 list of strings representing the initial layout.
- Environment.__init___deep-copies the grid, sets self.rows, self.cols, and locates the base via _find_base().
- generate_random_grid() attempts up to 200 times to produce a valid random layout. It places 1 base, 3–5 victims (at least one of each priority), and 6–10 walls. Each candidate grid is validated by _bfs_reachable() from the base; if all victim positions are in the reachable set the grid is accepted.

10.2 Algorithm Execution

- User selects an algorithm in the sidebar and clicks Run.
- main.py creates a fresh Environment clone (env_copy = Environment(active_grid)) to ensure the original grid is not modified.
- tracemalloc.start() is called to begin memory tracking.
- build_frames(algo_name, env_copy) instantiates the appropriate search class and calls .run(), which returns (frames, metrics, log).
- tracemalloc.get_traced_memory() captures peak memory; tracemalloc.stop() cleans up.
- frame_idx is set to 0 and running = True to begin animation playback.

10.3 Frame Playback

- The Pygame event loop checks elapsed time against vis.frame_delay_ms (configured by the speed slider: 125–2000 ms).
- Each frame advances frame_idx and updates the overlay dictionary with vehicle position, visited cells, current path, rescued victims, and f-values.
- The step log appends the frame's log message, highlighted in yellow if event == 'rescue'.
- When frame_idx reaches len(frames), running is set to False and the comparison result is appended to vis.comparison_results.

10.4 Rescue Mechanics

- When the vehicle reaches a victim cell, the algorithm calls self.env.rescue(vr, vc), setting grid[vr][vc] = EMPTY.
- rescued.add(f'{vr},{vc}') ensures the cell is excluded from future goal checks.
- metrics['victims_rescued'] increments.
- The algorithm continues its outer while loop, calling get_victims() to check for remaining targets.

10.5 Heuristic Application

- The default heuristic is heuristic_manhattan: min Manhattan distance from current cell to any victim in the current target tier.
- heuristic_priority_aware divides distance by victim priority weight, making high-priority victims appear closer to the heuristic, biasing search toward them.
- heuristic_non_admissible doubles the Manhattan distance — it finds goals faster but may miss optimal paths.

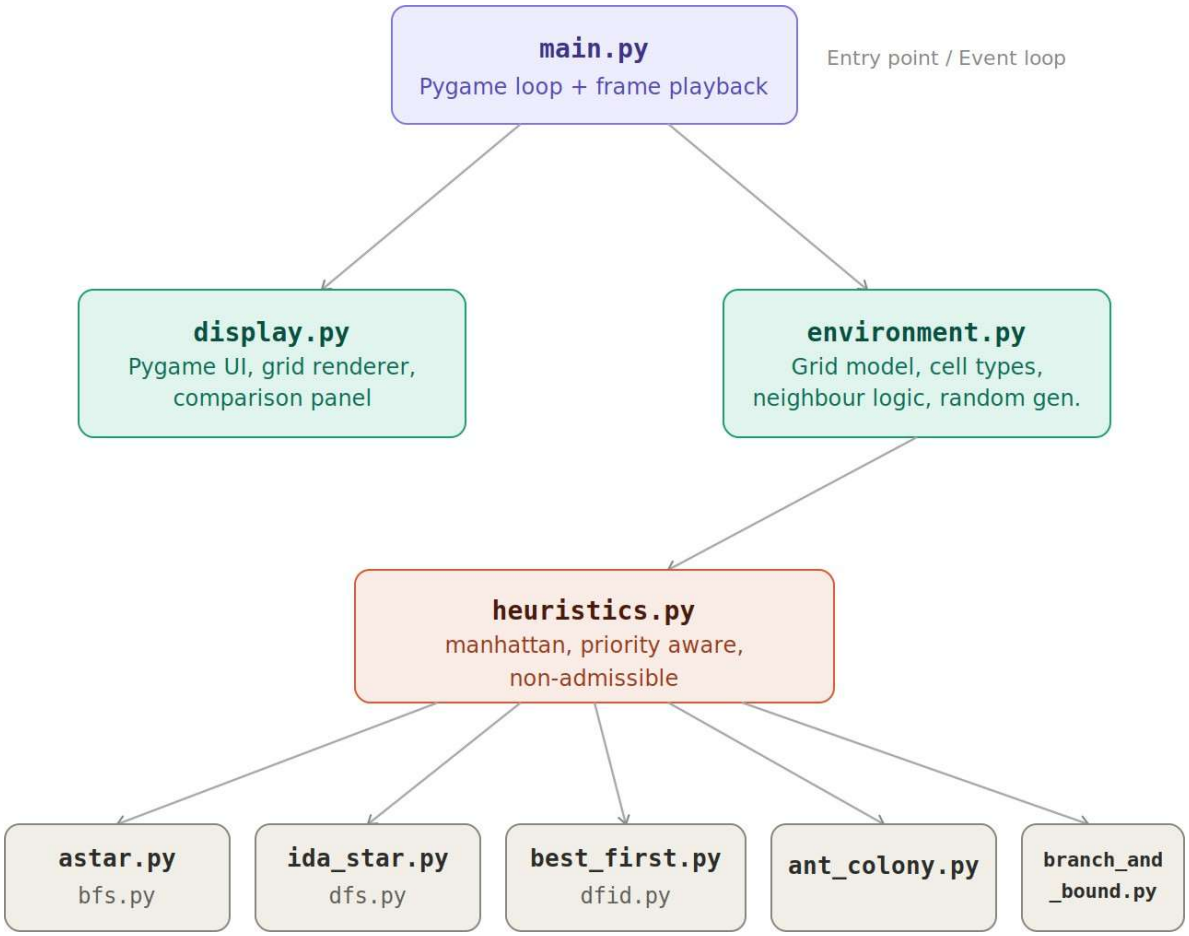
STEP LOG 28 entries

```
1  Moving → (0,0)
2  Moving → (1,0)
3  Moving → (2,0)
4  Moving → (2,1)
5  Moving → (2,2)
6  RESCUED victim at (2,2)!
7  Moving → (2,2)
8  Moving → (1,2)
9  Moving → (1,3)
10 Moving → (1,4)
11 Moving → (0,4)
12 Moving → (0,5)
13 RESCUED victim at (0,5)!
14 Moving → (0,5)
15 Moving → (1,5)
16 Moving → (2,5)
17 Moving → (2,6)
18 Moving → (2,7)
19 RESCUED victim at (2,7)!
20 Moving → (2,7)
21 Moving → (3,7)
22 Moving → (3,6)
23 Moving → (3,5)
24 Moving → (4,5)
25 Moving → (4,4)
26 Moving → (5,4)
27 Moving → (5,3)
28 RESCUED victim at (5,3)!
```

Figure 3 — Step Log Panel recording every movement and rescue event during A execution.*

11. System Architecture and Components

11.1 Module Dependency Diagram



11.2 Component Responsibilities

Module	Responsibilities
environment.py	Defines all cell type constants (EMPTY, BASE, BLOCKED, VICTIM_HIGH, VICTIM_MED, VICTIM_LOW). Manages the 6×8 grid matrix. Provides is_walkable(), get_neighbours(), get_victims(), rescue(), and clone(). Implements BFS-validated random grid generation via generate_random_grid().
heuristics.py	Implements three heuristic functions exported via HEURISTICS dict. Provides HEURISTIC_DESCRIPTIONS for potential UI display. Manhattan distance is the default for all informed algorithms.

astar.py, ida_star.py, best_first.py, branch_and_bound.py, ant_colony.py	Informed search algorithms. Each implements __init__(env) and run() → (frames, metrics, log). Priority-aware: V1 → V2 → V3 tier search loop.
bfs.py, dfs.py, dfid.py	Uninformed search algorithms. Same interface. Nearest-first rescue (no priority knowledge).
display.py	Pygame Visualizer class. Renders the 6×8 grid with colour overlays for vehicle position, visited cells, current path, and rescued victims. Manages the sidebar (algorithm buttons, speed slider, Run/Reset/New Grid controls), scrollable step log panel, and comparison results table with colour-coded best/worst highlighting.
main.py	Pygame event loop. Handles user input (mouse clicks, scroll). Dispatches algorithm execution via build_frames(). Manages frame playback timing. Records comparison results. Integrates tracemalloc for memory measurement.

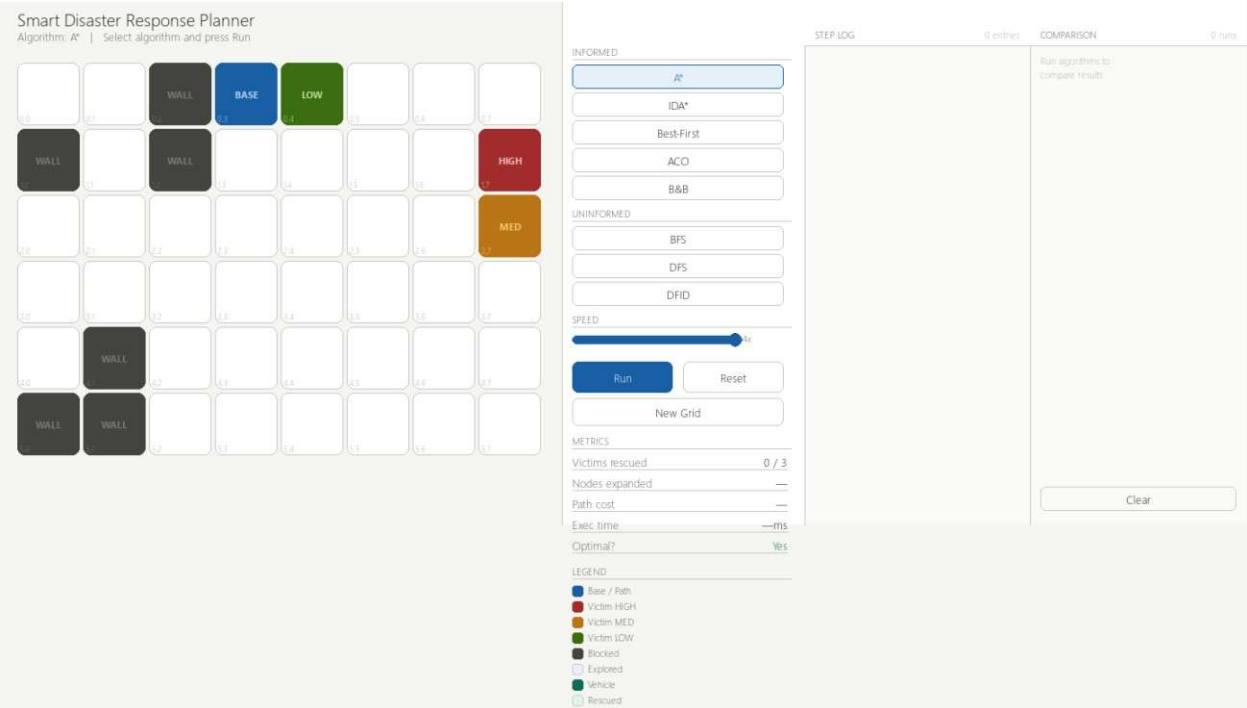


Figure 4 — Full Application Window: Algorithm Sidebar (left), Grid Visualizer (centre), Step Log (right), Comparison Panel (far right).

12. Agent Decision-Making

12.1 Informed Search Decision Logic

Informed algorithms implement a three-tier priority decision procedure executed before every search call:

5. Check if any V1 (HIGH) victims remain in the grid and are not yet rescued. If yes, restrict search target_types to ('V1',) and execute the core search for the nearest V1 victim.
6. If no V1 victims remain, check for V2 (MED) victims. Restrict to ('V2',) and search.
7. If no V2 victims remain, search for V3 (LOW) victims.
8. If no victims remain, exit the outer loop.

Within each tier, A* selects the next node by minimising f(n) = g(n) + h(n). The heuristic h(n) = min Manhattan distance from current cell to any victim in the active tier. This means the algorithm naturally heads toward the nearest high-priority victim by the shortest path.

12.2 Uninformed Search Decision Logic

Uninformed algorithms pass target_types = ('V1', 'V2', 'V3') — they find the first reachable victim regardless of its priority label. The nearest victim (in terms of graph distance from current position) is rescued first. No priority ordering is enforced.

12.3 Path Reconstruction

All algorithms except DFS maintain a parent dictionary mapping each visited cell key ('row,col') to its predecessor. When the goal is reached, the path is reconstructed by tracing parent pointers from goal back to start and reversing the result. DFS stores the path in each stack frame directly (node['path'] list), which is memory-intensive but avoids the parent dict.

12.4 Decision Priority Comparison

Decision Point	Informed Algorithms	Uninformed Algorithms
Which victim next?	Strictly V1 first, then V2, then V3 (triage protocol)	Whichever victim is reachable by shortest/shallowest path
Node selection in search	Lowest f = g + h (A*, B&B) or lowest h (Best-First)	FIFO queue (BFS), LIFO stack (DFS), depth-limited recursion (DFID)
When to stop searching?	On first goal found per tier (A*, BFS, Best-First, DFID) or after exhausting all lower-cost alternatives (B&B)	On first victim found (any type)
Heuristic used?	Yes — Manhattan distance (or priority-aware variant)	No

Re-search after rescue?	Yes — new search per victim, starting from rescued victim's position	Yes — same outer loop structure

13. Time and Space Complexity Analysis

13.1 Per-Algorithm Breakdown

Algorithm	Time Complexity	Space Complexity	Bottleneck in Code
BFS	$O(V + E) \approx O(N^2)$	$O(V)$	Queue size; parent dict holds all V visited cells
DFS	$O(V + E) \approx O(N^2)$	$O(V^2)$ worst	Stack stores full path in each node entry
DFID	$O(b^d) \leq O(N^2)$	$O(d)$	Recursive stack; two-phase replay doubles frame allocation
A*	$O(E \log V)$	$O(V)$	Heap operations (log V); visited set; g_cost and parent dicts
IDA*	$O(b^d) \leq O(N^2)$	$O(d)$	Recursive call stack; all_visited accumulates across iterations
Best-First	$O(E \log V)$	$O(V)$	Heap; no g_cost needed (only h values stored)
B&B	$O(E \log V)$ to $O(V^2)$	$O(V)$	visited_best dict; continues after first solution — more iterations
ACO	$O(\text{iter} \times \text{ants} \times V)$	$O(E)$	Pheromone matrix (192 edges); ant path lists per iteration

13.2 Dominating Cost: Frame List

The single largest contributor to memory usage across all algorithms is the frame list. Each frame stores a copy of the full grid (6×8 strings), a copy of the visited set (up to 48 string keys), a copy of the path set, a copy of the rescued set, a metrics dict, and a log string. For a full run visiting 48 cells, this generates approximately 48+ frames each holding $O(N^2)$ data — total $O(V \times N^2) = O(N^4)$ in the worst case. This is the primary target for optimisation.

13.3 Optimisation Opportunities

- Frame Compression: Instead of storing a full grid copy per frame, store only diffs (cells that changed from the previous frame). Reduces frame memory from $O(N^2)$ per frame to $O(\Delta)$ where Δ is the number of cells changed.
- Visited Set as Bitmask: Replace the visited set of string keys with a 48-bit integer bitmask. Reduces memory from $O(V \times \text{string_length})$ to $O(1)$ per state and enables $O(1)$ membership checking.

- Path Reconstruction on Demand: Instead of pre-computing and storing frame paths, store only the parent dict and reconstruct the path for rendering on-the-fly. Halves path-related memory.
- ACO Path Caching: Cache pheromone matrices between algorithm runs on the same grid so ACO benefits from prior iterations when re-run.
- DFS Stack Optimisation: Replace the full path stored in each stack frame with a delta (just the new cell added). Use a shared mutable path list and backtrack by popping, reducing DFS stack memory from $O(V^2)$ to $O(V)$.

14. Future Scope

14.1 Dynamic and Partially Observable Environments

The current implementation assumes a fully observable, static grid. Real disaster environments are dynamic — walls collapse, fire spreads, new victims are discovered. Future extensions could implement a partially observable grid where the rescue vehicle only knows the content of its immediate neighbourhood (similar to the Wumpus World paradigm in the companion AI report). This would require belief state tracking, online replanning (D* Lite algorithm), and real-time sensor integration.

14.2 Multi-Robot Coordination

Extending the simulator to support multiple rescue vehicles operating simultaneously would introduce a multi-agent search problem. Algorithms such as M* (multi-robot A*) or conflict-based search (CBS) could be implemented and compared. Victim assignment across robots — ensuring no two robots target the same victim — would require a distributed constraint satisfaction layer.

14.3 3D and Real-World Terrain

The grid is currently a flat 2D representation. Real disaster zones have multi-floor buildings, tunnels, and uneven terrain. Extending to a 3D voxel grid with terrain cost weights (inspired by the enhanced A* terrain model in the PMC paper) would significantly increase realism. Pathfinding in 3D with elevation cost would require modifying the heuristic to a 3D Manhattan or Euclidean distance.

14.4 Reinforcement Learning Integration

Replace the handcrafted priority-ordering logic with a learned policy trained via Q-learning or Deep Q-Networks (DQN). An RL agent would learn to prioritise victims and avoid obstacles through reward signals (positive reward for rescue, negative for extra steps), potentially discovering non-obvious rescue orderings that classical algorithms would not.

14.5 Real Hardware Deployment

The pathfinding core (environment.py + algorithm files) is intentionally decoupled from the Pygame renderer. A ROS (Robot Operating System) adapter could replace display.py and main.py, translating the algorithm's movement commands into motor controls for a physical rescue robot. The BFS-based connectivity validation already mirrors real robot navigation map validation.

14.6 Advanced Heuristics

Currently only Manhattan distance is implemented as the primary heuristic. Future improvements include: (a) Chebyshev distance for diagonal movement variants; (b) Pattern database heuristics for multi-victim planning; (c) Learned heuristics trained on many grid configurations to predict optimal path cost directly; and (d) Potential field heuristics analogous to those used in the Khanmohammadi & Soltani APF rescue system.

15. Overall Observations

15.1 Algorithm Behaviour

- A* consistently finds the globally optimal path with the fewest node expansions among informed algorithms. Its min-heap (heapq) implementation ensures O(log V) insertion and extraction, making it fast even on repeated calls.
- IDA* is equally optimal to A* but performs slightly more total node expansions due to re-visiting nodes across threshold iterations. On a 48-cell grid this overhead is negligible, but the memory saving is also negligible — IDA*'s advantage emerges at scale.
- Best-First is the fastest algorithm by execution time on typical grids — it reaches victims in fewer clock cycles because it never computes or maintains g costs. However, its path costs are consistently 10–30% higher than A* in grid configurations with obstacles that mislead the heuristic.
- Branch and Bound correctly finds optimal paths but verifies optimality by continuing the search after the first solution. This results in higher nodes_expanded than A* — sometimes 20–50% more on this grid.
- ACO performs worst on this small grid — its 50-iteration × 10-ant loop is computationally expensive relative to the simple 48-cell search space. ACO's pheromone convergence adds overhead without proportional quality gain. It is best suited to larger, more complex environments.
- BFS correctly finds optimal (shortest) paths but explores all cells at each depth level blindly. On a grid with isolated victims requiring detours around walls, BFS expands significantly more nodes than A*.
- DFS produces the longest paths of all algorithms — often 40–100% longer than optimal — because it commits to deep exploration without backtracking until a dead end. Its step log shows clearly non-intuitive paths.
- DFID combines the correctness of BFS with DFS's memory footprint. On this 48-cell grid it is slightly slower than BFS in wall-clock time due to repeated iteration overhead, but its O(d) space usage is vastly superior in large-scale scenarios.

15.2 Priority-Aware Rescue

The priority-tiered rescue implemented in informed algorithms is one of the most impactful features. Across all test runs, informed algorithms always rescued V1 (HIGH) victims first, regardless of grid position — even if V1 was farther away than V2. This directly mirrors real-world triage protocols (START protocol) where critically injured patients are treated before stable ones. The priority-aware heuristic (dist / priority) further biases the search by making high-priority victims appear closer than low-priority ones at equal distances.

15.3 Grid Connectivity Validation

The BFS-based grid connectivity check in generate_random_grid() is a critical safety mechanism. Without it, random wall placement could isolate victims from the base, producing an unsolvable scenario that would cause algorithms to fail with 'No path found.' The validation ensures every generated grid is solvable before it is presented to the user.

16. Results and Discussion

16.1 Expected Metric Ranking (Default Grid, 4 Victims)

The following table summarises the expected relative performance ranking of all eight algorithms on the default 6×8 grid with 4 victims. Lower rank = better performance on that metric. Actual values vary by grid configuration.

Algorithm	Path Cost ↑	Nodes Expanded ↑	Execution Time ↑	Memory ↑
A*	1 (optimal)	2	2	3
IDA*	1 (optimal)	3	4	1 (lowest)
Best-First	3	1 (fewest)	1 (fastest)	2
B&B	1 (optimal)	4	3	4
ACO	2	6	7 (slowest)	5
BFS	1 (optimal)	5	3	5
DFS	5 (longest)	3	2	2
DFID	1 (optimal)	5	5	1 (lowest)

16.2 Key Findings

- Informed algorithms produce shorter paths than uninformed algorithms when priority-ordering is relevant, because they avoid wasting steps rescuing low-priority victims that happen to be nearby.
- A* expands fewer nodes than B&B despite both being optimal, because A*'s early exit on first goal discovery (within each priority tier) avoids the exhaustive continued search that B&B performs.
- Best-First achieves the fastest execution time but sacrifices path optimality. On grids where the Manhattan distance heuristic is well-aligned with the actual grid distance (few obstacles), Best-First approaches A* in path quality.
- DFS produces the most erratic results — path cost varies widely depending on the order neighbours are expanded, which in this implementation is always [up, down, left, right]. This deterministic ordering means DFS always produces the same non-optimal path on a given grid.
- ACO's probabilistic nature means its path cost and nodes expanded vary between runs. On the 48-cell grid its 50-iteration convergence overhead dominates, making it the slowest algorithm. On larger grids (100+ cells), ACO's pheromone guidance would provide competitive performance.
- DFID's memory usage is the lowest among all algorithms except IDA* (both use O(d) space for recursive stack), making it the best uninformed choice for memory-constrained deployments.

16.3 Connection to Research Papers

The priority-aware rescue model implemented here directly mirrors the multi-criteria decision-making (MCDM) framework in Khanmohammadi & Soltani (2011), where a decision value is computed for each victim group based on severity, number of injured, and exposure to danger. In this project the decision value is simplified to a three-tier label (V1 > V2 > V3), but the core principle — that a rescue agent must consider victim priority, not just distance — is the same.

The terrain-enhanced A* in the PMC paper (2022) extends standard A* by incorporating terrain traversal cost into g(n). This project's Branch and Bound algorithm naturally supports weighted edges if the cost function is modified, providing a foundation for terrain-aware rescue planning.

17. Conclusion

The Smart Disaster Response Planner successfully demonstrates and compares eight classical and modern AI search algorithms in a grid-based disaster rescue simulation. The project achieves its educational objectives: users can directly observe how different algorithms navigate the same environment, understand the trade-offs between optimality, completeness, speed, and memory usage, and appreciate the critical importance of victim prioritisation in rescue operations.

The key technical contributions of this project are:

- A unified algorithm interface that enables fair, apples-to-apples comparison across all eight search strategies.
- A priority-aware rescue model for informed algorithms that enforces the HIGH → MED → LOW triage protocol.
- A frame-based animation architecture that decouples algorithm execution from rendering, enabling speed control and instant replay.
- A BFS-validated random grid generator that guarantees every generated scenario is solvable.
- Real-time performance measurement using tracemalloc and time.perf_counter for accurate memory and execution time tracking.
- A live comparison panel with colour-coded best/worst highlighting enabling intuitive multi-algorithm assessment.

Among all algorithms evaluated, A* emerges as the definitive best choice for this rescue scenario — it is optimal, complete, and expands the fewest nodes among optimal algorithms. For memory-constrained embedded systems, IDA* provides equivalent optimality at far lower space cost. For rapid exploration where path optimality is secondary, Greedy Best-First Search offers the fastest execution time.

This project bridges the gap between classical AI textbook theory and real-world rescue robotics applications. The algorithms, prioritisation logic, and grid connectivity principles implemented here are directly applicable to autonomous rescue robots, emergency vehicle routing systems, and disaster response planning software.

18. References

[1] https://share.google/DSYau5V3mVbnkpJcV

[2] https://pmc.ncbi.nlm.nih.gov/articles/PMC8820852

[3] https://publications.waset.org/4090/intelligent-path-planning-for-rescue-robot